

Rust + Actix Web: Partnering to Build Ultra-Fast APIs

Rust + Actix Web is a combo that always shines when building ultra-fast APIs. If you are coming from Node.js, Python, or even Go, the jump in performance can seem dramatic – because it is.



Actix Web is often in the top frameworks in the TechEmpower Framework Benchmarks, sometimes even competing directly against low-level C/C++ frameworks. The reason is simple; it is based on Rust's zero cost abstractions and asynchronous non-blocking runtime.

```
use actix_web::{get, App, HttpServer, Responder};
#[get("/")]
async fn hello() -> impl Responder {
    "Hello, fast world!"
}
#[actix_web::main]
async fn main() -> std::io::Result<> {
    HttpServer::new(|| App::new().service(hello))
        .bind(("127.0.0.1", 8080))?
        .run()
        .await
}
```

Environmental setup and minimalist tooling

If you want to create ultra-fast APIs, your environment should be as efficient as your runtime. Rust encourages a

‘sovereign’ and lean toolchain -- maximum control, minimum dependencies. First, you need to install Rust with *rustup*, the official toolchain manager. It allows you to manage compiler versions, targets and components cleanly:

```
curl https://sh.rustup.rs -sSf | sh
rustup update
rustup component add clippy rustfmt
```

Once installed, cargo becomes a command centre. It handles testing, building, and dependency management:

```
cargo new ultra_api
cd ultra_api
cargo run
```

Next, focus on binary optimisation. Rust binaries can grow due to static linking; also, tools like cargo-bloat help you analyse what's taking space:

```
cargo install cargo-bloat
cargo bloat --release
```